

National Test Project

Module A Functionality

Day 1 P.M.

Mobile Applications Development

Contents

Introduction	3
Description of project and tasks	3
General Demand	6
Page Demand	7
Instruction to the competitor	13
Web API documents	13

Introduction

CitiMove is a Government of India initiative to build Smart City mobility applications for major urban centres. As part of the IndiaSkills national competition, you are required to build a mobile transit companion app named "MyCity Transit".

The app helps citizens of smart cities navigate public transport options, explore city services, and receive real-time bus/metro updates — all from a single interface.

As a Mobile Application Developer, your task is to build a fully functional application based on the provided wireframe descriptions and the specific development requirements outlined in this document.

Description of project and tasks

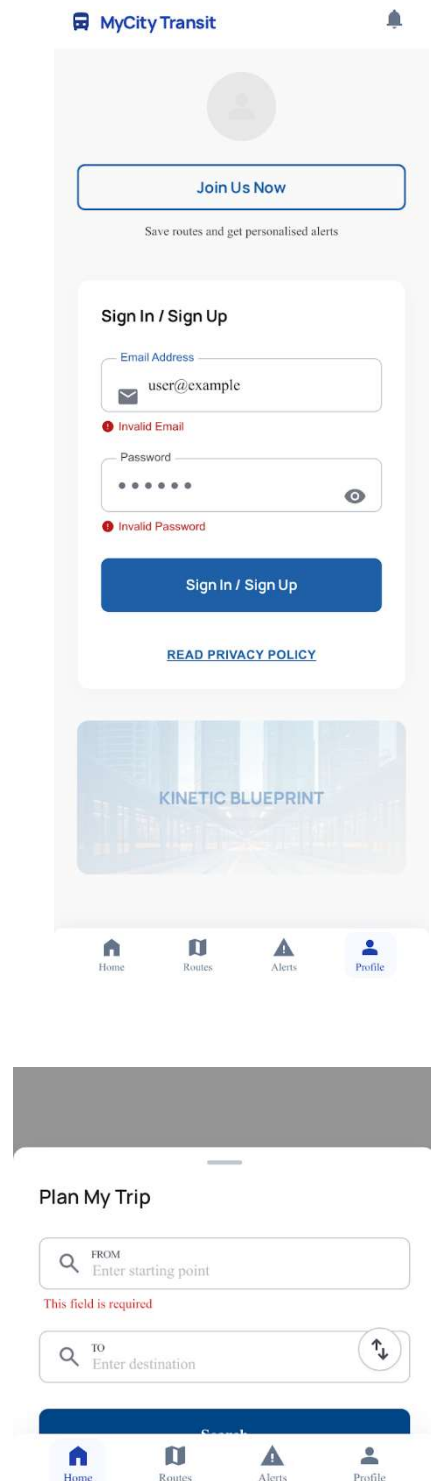
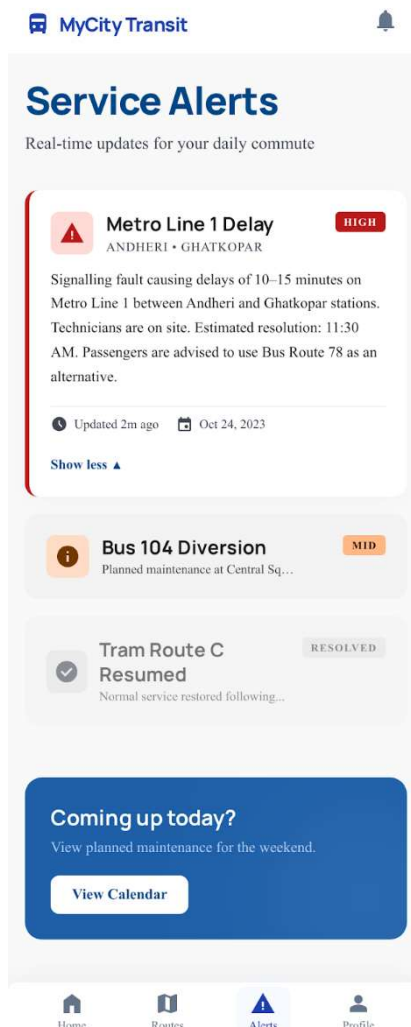
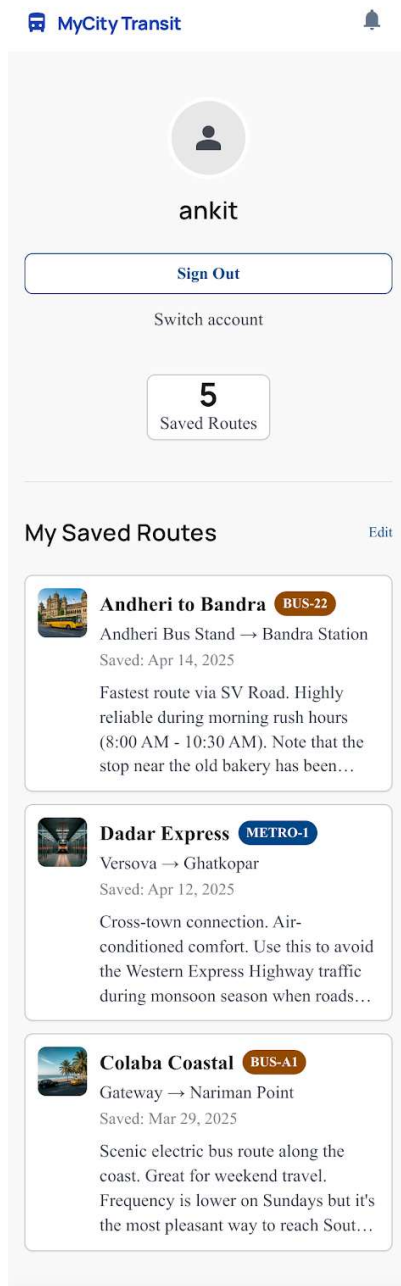
You do not need to precisely replicate the visual aesthetics of the wireframes — layout can differ. However, all functional requirements must be implemented correctly and the app must be easy to use and navigate.

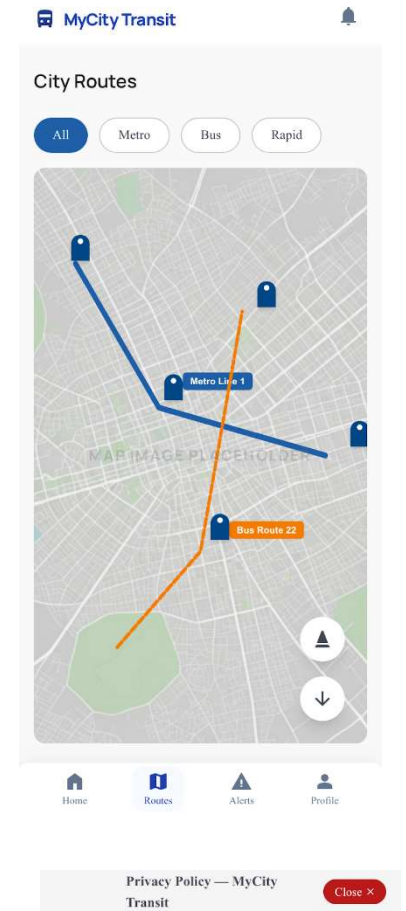
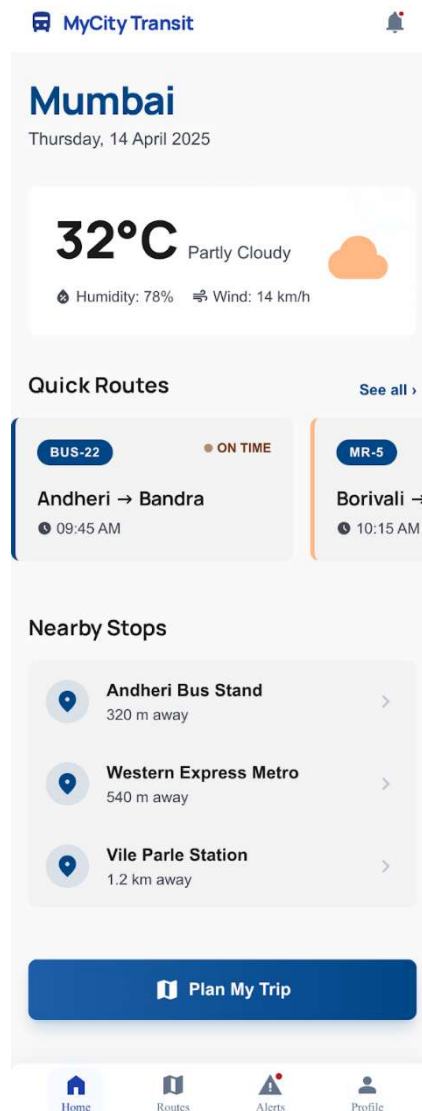
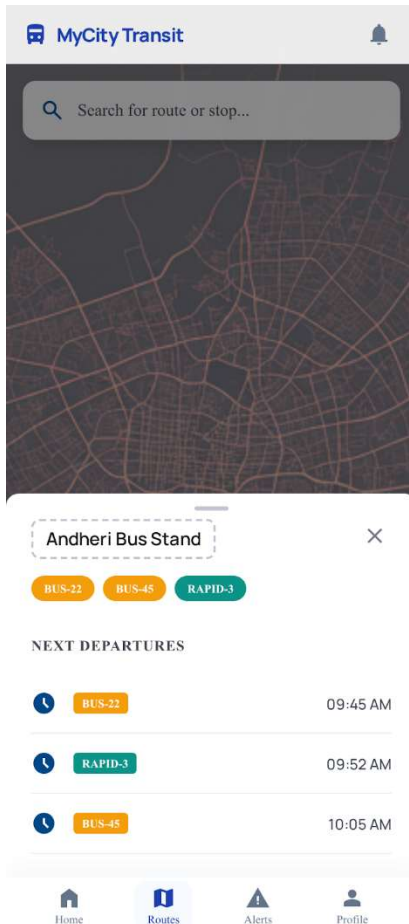
PART	NAME	COMPETITION TIME	MARKING DEVICES
Day 1 P.M.	Functionality	3 hours	Mobile device

Below is an overview of the wireframes. Please check the media-files for details.

The application consists of four main screens:

- Home Screen – Live city dashboard with weather and transit summary
- Routes Screen – Interactive map with bus and metro routes
- Alert Screen – Real-time disruption and service alerts
- Profile Screen – User account with saved routes





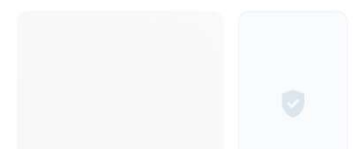
Privacy Policy

Last updated: April 2025

Data We Collect

How We Use Your Data

Your Rights



General Demand

1. Build and run the application fully according to all functional requirements in this document.
2. The application must display the application icon using the file "art_icon_citymove.png" from the provided media-files.
3. The application name shown in the operating system must be "MyCity Transit".
4. All screens must include a bottom navigation bar with four tabs.
5. Bottom navigation bar:
 - App logo/icon is displayed at the top of each page header.
 - "Home" tab with icon — navigates to the Home screen.
 - "Routes" tab with icon — navigates to the Routes screen.
 - "Alerts" tab with icon — navigates to the Alerts screen.
 - "Profile" tab with icon — navigates to the Profile screen.
 - The active tab must be visually highlighted (different colour or underline).
6. The application must support both Android and iOS.
7. All data must be fetched from the provided Web API endpoints.

Page Demand

Home Page

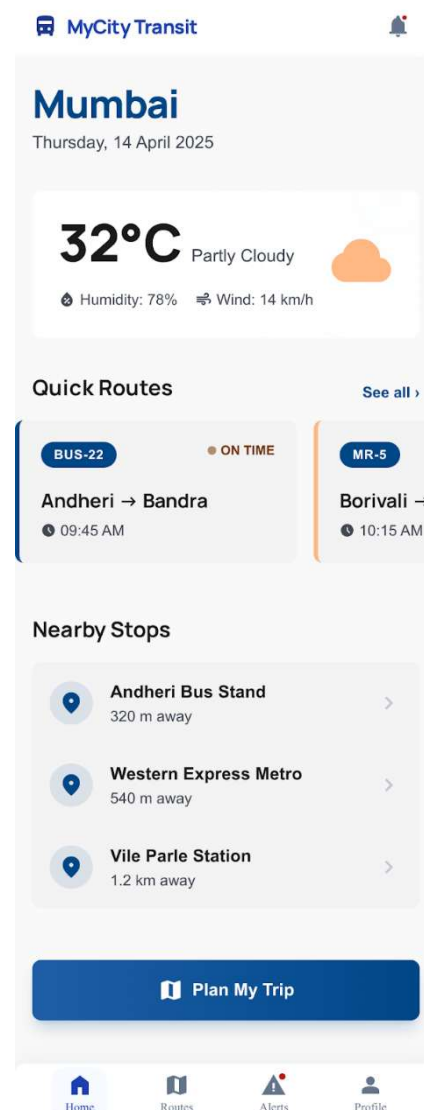
The Home screen is the default landing screen of the app. It shows a real-time city dashboard.

Element include:

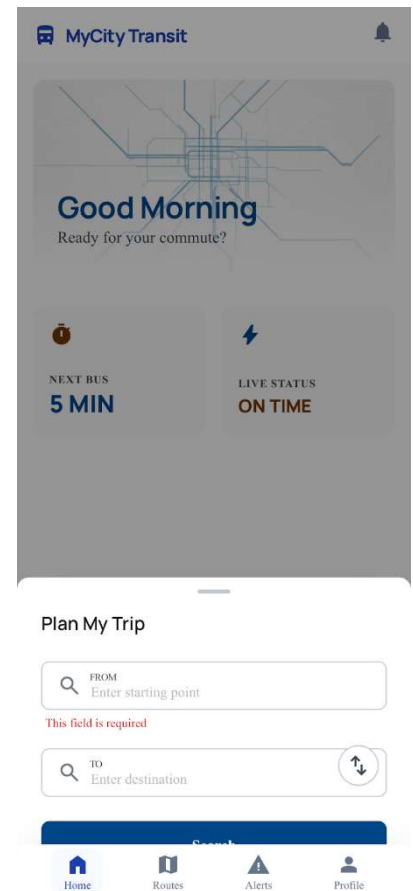
1. Header with app name "MyCity Transit" and app icon.
2. City name and current date are prominently displayed.
3. Live weather widget:
 - City name
 - Current temperature (in °C)
 - Weather condition icon (Sunny / Cloudy / Rainy)
 - Humidity and wind speed
4. "Quick Routes" horizontal scrollable card list, each card showing:
 - Route number (e.g., BUS-22, METRO-3)
 - Route name (e.g., Andheri → Bandra)
 - Estimated next departure time
 - A coloured badge: Green (On Time), Yellow (Delayed), Red (Cancelled)
5. "Nearby Stops" section listing the 3 closest transit stops with distance in metres.
6. A "Plan My Trip" prominent button.

Functional requirements:

1. Transit route data is fetched from the Web API on screen load.
2. Weather data is fetched from the Web API and displayed in the live weather widget.
3. Quick Routes list is horizontally scrollable.
4. Tapping a Quick Route card navigates to the Routes screen and highlights that route on the map.
5. The status badge colour updates based on the API response field: "on_time", "delayed", or "cancelled".
6. Tapping "Plan My Trip" opens a bottom sheet modal with:
 - "From" location input field with placeholder "Enter starting point"



- "To" location input field with placeholder "Enter destination"
 - A "Search" button
7. When "Search" is tapped:
- If either field is empty, display a red inline error: "This field is required"
 - If both fields are filled, navigate to the Routes screen displaying the matching route
8. The "Nearby Stops" section displays data fetched from the API, sorted by distance ascending.
9. The Home screen must auto-refresh transit data every 60 seconds.

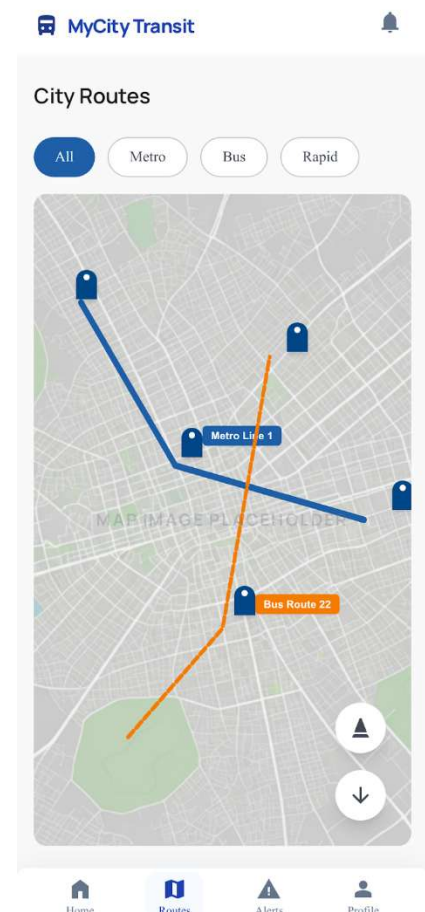


Routes Screen

The Routes screen shows an interactive map with overlaid transit routes and stops markers.

Element include:

1. Page title "City Routes".
2. Zoomable and pannable map view.
3. Route overlay lines on the map (coloured by transport type: Blue for Metro, Orange for Bus).
4. Stop marker icons on the map.
5. Route filter chips: "All", "Metro", "Bus", "Rapid".
6. Floating "Save Map" button (download icon).
7. Route detail bottom sheet (when a stop marker is tapped):
 - Stop name
 - Available routes passing through this stop
 - Next 3 departure times
 - "Navigate" underlined text button
 - Close icon button



Functional requirements:

1. Route and stop data is loaded from the provided JSON file bundled in the app assets.
2. The map supports pinch-to-zoom and one-finger pan after zooming.
3. Stop marker icons maintain correct position and fixed size as the map is scaled.
4. Tapping a filter chip filters the visible routes and markers on the map by transport type.
5. Route overlay lines always remain correctly scaled and positioned on the map.
6. Tapping the "Save Map" button saves a screenshot of the current map view (with overlays) to the device gallery.
7. Tapping a stop marker opens the route detail bottom sheet.
8. Tapping "Navigate" opens the device's default Maps application (Google Maps / Apple Maps) to the stop's coordinates.
9. The stop name in the route detail sheet can be long-pressed and dragged into a messaging app when the device is in split-screen mode.
10. Tapping the close button dismisses the bottom sheet.

Alerts Screen

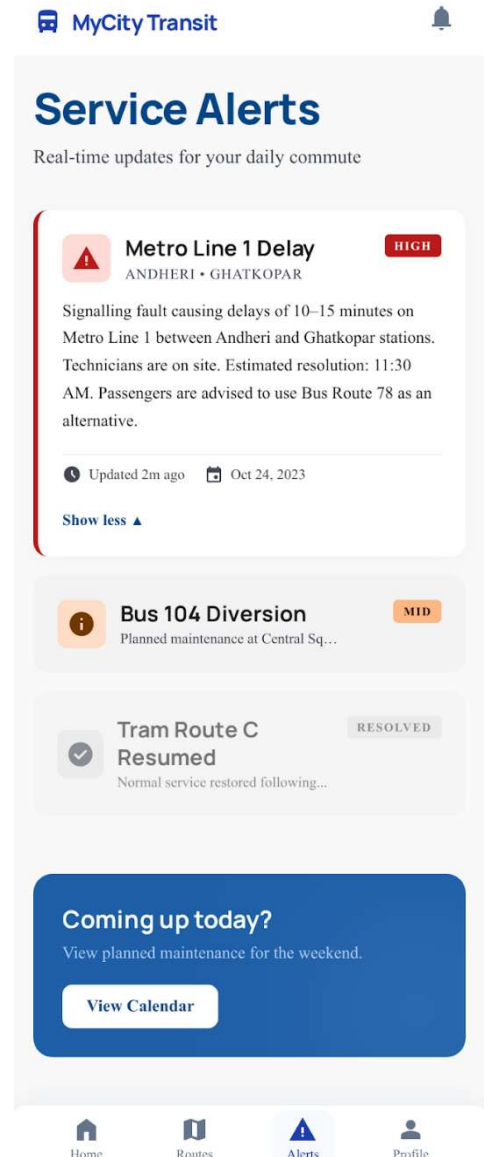
The Alerts screen displays real-time service disruptions, planned maintenance, and notifications relevant to the user.

Element include:

1. Page title "Service Alerts".
2. Filter tabs: "All", "Active", "Planned", "Resolved".
3. Alert cards list, each card showing:
 - Alert type icon (e.g., warning triangle, info circle)
 - Alert title
 - Affected route(s)
 - Date and time of the alert
 - Short description (max 2 lines, truncated with "...")
 - Severity badge: "High" (Red), "Medium" (Orange), "Low" (Grey)
4. A "Mark All Read" text button at the top right.
5. Empty state illustration when no alerts are present for the selected filter.

Functional requirements:

1. Alert data is fetched from the Web API.
2. The list is sorted by severity (High first) and then by date (newest first).
3. Tapping a filter tab filters the list by alert status.
4. Unread alerts are visually distinguished (e.g., bold title or coloured left border).
5. Tapping an alert card expands it in-line to show the full description text (toggle expand/collapse).
6. Tapping "Mark All Read" marks all alerts as read and removes the unread visual indicator.
7. New alerts received since the last refresh display a red dot badge on the Alerts bottom tab icon.
8. The alert date-time must be displayed in the format: "Mon DD, YYYY HH:MM AM/PM" (e.g., "Apr 14, 2025 09:30 AM").



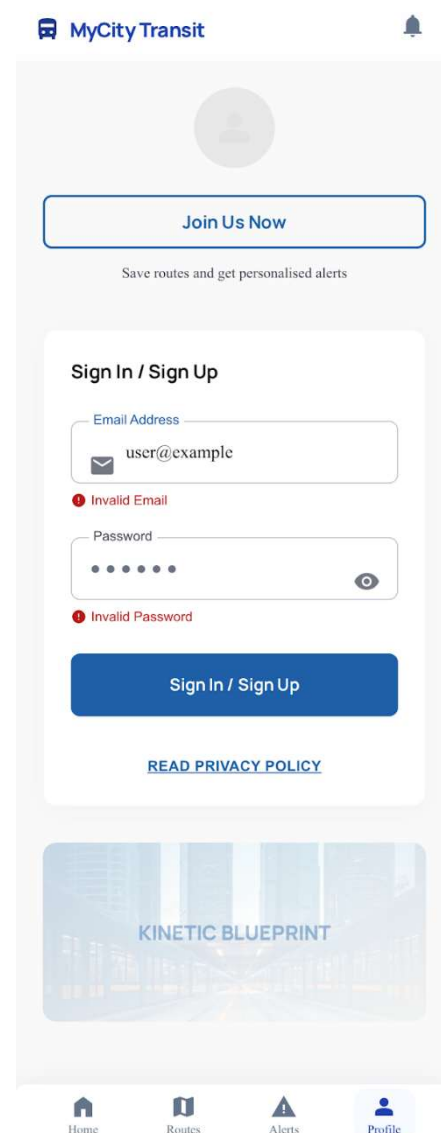
Profile Screen (Before Sign In)

Element include:

1. User avatar placeholder icon.
2. Button: "Join Us Now".
3. Text: "Save routes and get personalised alerts".
4. Sign In / Sign Up area (hidden by default):
 - Title "Sign In / Sign Up"
 - Email input field with placeholder "Your Email Address"
 - Password input field with placeholder "Your Password" and an eye icon button
 - Button: "Sign In / Sign Up"
 - Blue underlined link: "READ PRIVACY POLICY"

Functional requirements:

1. Authentication uses the Web API.
2. The Sign In / Sign Up area is hidden by default.
3. Tapping "Join Us Now" reveals the Sign In / Sign Up area.
4. Email field validates format in real time:
 - Invalid format → show red message "Invalid Email"
 - Valid format → no message shown
5. Password field validates in real time (minimum 6 characters, must include letters and numbers):
 - Invalid → show red message "Invalid Password"
 - Valid → no message shown
6. Password is masked by default.
7. Holding the eye icon reveals the password, releasing masks it again.
8. Tapping "READ PRIVACY POLICY" opens a WebView dialog displaying the privacy policy page with CSS correctly applied.
9. The WebView dialog can only be closed using the close button on the privacy policy page itself.
10. Failed sign-in shows a system notification with the API error reason.
11. Successful sign-in plays the sound file "sign_in_success.mp3".
12. After successful sign-in, the screen transitions to the Profile (After Sign In) view.



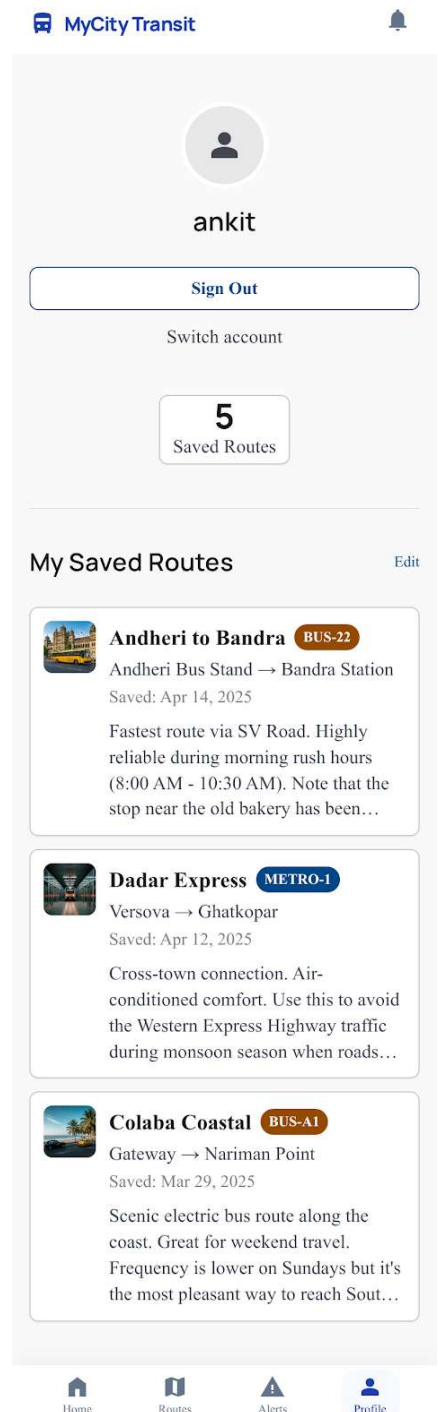
Profile Screen (After Sign In)

Elements include:

1. User avatar icon.
2. Username derived from the email address (text before "@").
3. "Sign Out" button.
4. Text: "Switch account".
5. Saved routes count: "XX Saved Routes".
6. "My Saved Routes" section heading.
7. Saved routes list, each item showing:
 - Route icon/thumbnail
 - Route name and number
 - Start and end stops
 - Date saved
 - Short description (max 4 lines, truncated with "...")

Functional requirements:

1. Username is extracted from the email address (the part before "@").
2. Saved routes count reflects the actual number of saved routes.
3. The "My Saved Routes" list shows all routes the user has saved.
4. Newly saved routes appear at the top of the list.
5. Each route item shows a maximum of 4 lines of description; excess text is truncated with "...".
6. Tapping "Sign Out" clears the session and returns to the Profile (Before Sign In) view.



Instruction to the competitor

1. Please create the project with correct package name (Android).
2. You should save the project in the folder XX_Module_A_PM;
3. You should rename the generated apk (Android) file as XX_Module_A_PM.apk (Android) and save it in the root of the XX_Module_A_PM folder;
4. The whole XX_Module_A_PM folder should be pushed to the provided remote Git repository in time;
5. Note: "XX" is your workstation code.

Web API documents

[USER] SIGN IN OR SIGN UP				
No.	1	Method	POST	URL: http://{{BASE_URL}}:{{PORT}}/api/users/signin
Request				
Key	Value (Example)	Description		
userEmailAddress	user@email.com	Valid email address		
userPassword	pass123	Minimum 6 characters including letters and numbers		
Successful Response Body (Example)				
<pre>{ "msg": "Sign in successful", "data": { "auth_token": "ABCD1234EFGH5678IJKL" } }</pre>				

[TRANSIT] GET ALL ROUTES				
No.	2	Method	GET	URL: http://{{BASE_URL}}:{{PORT}}/api/transit/routes
Successful Response Body (Example)				
{				

[TRANSIT] GET ALL ROUTES

```
{
  "msg": "Success",
  "data": [
    {
      "route_id": "RTE-BUS-022",
      "route_name": "Andheri to Bandra",
      "route_type": "bus",
      "status": "on_time",
      "next_departure": "2025-04-14 09:45:00",
      "stops": ["Andheri", "Vile Parle", "Santacruz", "Bandra"]
    }
  ]
}
```

[WEATHER] GET CURRENT WEATHER

No.	3	Method	GET	URL: http://{{BASE_URL}}:{{PORT}}/api/weather/current
-----	---	--------	-----	--

Successful Response Body (Example)

```
{
  "msg": "Success",
  "data": {
    "city": "Mumbai",
    "temperature_c": 32,
    "condition": "Partly Cloudy",
    "humidity_pct": 78,
    "wind_kmh": 14
  }
}
```

[ROUTE] SAVE A ROUTE

No.	4	Method	PUT	URL: http://{{BASE_URL}}:{{PORT}}/api/routes/save
-----	---	--------	-----	--

Request

Key	Value (Example)	Description
route_id	<i>RTE-BUS-022</i>	The ID of the route to save

Successful Response Body (Example)

```
{
  "msg": "Success",
  "data": {
    "route_id": "RTE-BUS-022",
  }
}
```

[ROUTE] SAVE A ROUTE

```
{
  "saved_at": "2025-04-14 10:05:00"
}
```

Note: The PUT /api/routes/save endpoint requires the auth_token header:

Key	Value (Example)	Description
auth_token	ABCD1234EFGH5678IJKL	The token returned after successful sign-in

[ALERTS] GET SERVICE ALERTS

No.	5	Method	GET	URL: http://{{BASE_URL}}:{{PORT}}/api/alerts
-----	---	--------	-----	--

Successful Response Body (Example)

```
{
  "msg": "Success",
  "data": [
    {
      "alert_id": "ALT-001",
      "title": "Metro Line 1 Delay",
      "affected_routes": ["METRO-1"],
      "status": "active",
      "severity": "high",
      "description": "Signalling fault causing delays of 10-15 minutes.",
      "created_at": "2025-04-14 08:30:00"
    }
  ]
}
```

[RESOURCES] GET STATIC RESOURCE FILE

No.	6	Method	GET	URL: http://{{BASE_URL}}:{{PORT}}/api/{{path}}
-----	---	--------	-----	--

Request

Key	Value (Example)	Description
Relative path	resources/maps/mumbai_base.png	Relative path to map images, icons, or route JSONs

[RESOURCES] PRIVACY POLICY WEBPAGE				
No.	7	Method	GET	URL: http://{{BASE_URL}}:{{PORT}}/api/privacy-policy

[ROUTE] GET MY SAVED ROUTES				
No.	8	Method	GET	URL: http://{{BASE_URL}}:{{PORT}}/api/routes/saved

Successful Response Body (Example)

```
{
  "msg": "Success",
  "data": [
    {
      "route_id": "RTE-BUS-022",
      "route_name": "Andheri to Bandra",
      "saved_at": "2025-04-14 10:05:00"
    }
  ]
}
```

Note: The GET /api/routes/saved endpoint also requires the auth_token request header (same as API #4).